

Pooleno Senior Node.js Backend Challenge

Resilient Wallet Reservation and Trade Execution

Candidate profile alignment

This challenge is designed around the candidate's stated experience with Node.js, TypeScript, RabbitMQ, Redis, MongoDB, PostgreSQL, Prometheus, Docker, real-time systems, distributed services, and exchange-related platforms. It intentionally focuses on backend architecture, consistency, messaging, and production readiness rather than frontend development.

1. Objective

Design and implement a small backend service for a simplified digital-asset conversion flow.

The challenge must demonstrate:

- Domain-Driven Design
- Correct handling of wallet balance and concurrent requests
- Idempotent command processing
- Event-driven communication
- Clear application and domain boundaries
- Practical use of AI during software development
- Deliberate prompt architecture and prompt engineering
- Production-oriented testing and observability

The expected implementation time is approximately **15 to 18 hours**.

The solution should remain intentionally narrow. Do not build a complete exchange platform.

2. Business Scenario

A user wants to convert one digital asset into another.

Example:

Convert 100 USDT to BTC

The flow is:

1. The user requests a price quote.
 2. The quote is valid for 20 seconds.
 3. The user accepts the quote.
 4. The required source asset must be reserved in the user's wallet.
 5. A trade execution request is published asynchronously.
 6. A simulated external exchange processes the trade.
 7. The conversion is either completed or marked as failed.
 8. Reserved funds must never be spent by another concurrent request.
 9. Repeated client requests or repeated message delivery must not create duplicate trades.
-

3. Required Scope

Implement the following capabilities only:

3.1 Create Quote

Create a quote for converting one asset to another.

Example request:

```
POST /quotes
{
  "userId": "user-123",
  "sourceAsset": "USDT",
  "targetAsset": "BTC",
  "sourceAmount": "100"
}
```

Example response:

```
{
  "quoteId": "quote-001",
  "sourceAsset": "USDT",
  "targetAsset": "BTC",
  "sourceAmount": "100",
  "targetAmount": "0.00161",
  "rate": "0.0000161",
  "expiresAt": "2026-08-01T10:00:20Z",
  "status": "ACTIVE"
}
```

The price may be generated using a deterministic fake pricing provider.

No real exchange integration is required.

3.2 Accept Quote

Accept an active quote.

POST /quotes/{quoteId}/accept

Required header:

Idempotency-Key: 4a985cf8-4f46-49b4-a916-3744073b3794

Acceptance must:

- Validate that the quote exists.
 - Validate that the quote has not expired.
 - Validate that it has not already been accepted.
 - Atomically reserve the required wallet balance.
 - Prevent the wallet from becoming negative.
 - Mark the quote as accepted.
 - Create a conversion operation.
 - Persist an outbox message in the same database transaction.
-

3.3 Execute Conversion Asynchronously

An asynchronous worker must publish and process a trade execution event.

Example event:

```
{
  "eventId": "event-001",
  "eventType": "ConversionExecutionRequested",
  "conversionId": "conversion-001",
  "userId": "user-123",
  "sourceAsset": "USDT",
  "targetAsset": "BTC",
  "sourceAmount": "100",
  "targetAmount": "0.00161",
  "occurredAt": "2026-08-01T10:00:05Z"
}
```

The simulated execution handler must:

- Process the event idempotently.
- Simulate either success or failure.
- Complete or release the wallet reservation.
- Prevent duplicate balance updates.
- Store the final conversion state.

3.4 Query Conversion Status

GET /conversions/{conversionId}

Example response:

```
{
  "conversionId": "conversion-001",
  "status": "COMPLETED",
  "sourceAsset": "USDT",
  "targetAsset": "BTC",
  "sourceAmount": "100",
  "targetAmount": "0.00161",
  "createdAt": "2026-08-01T10:00:05Z",
  "completedAt": "2026-08-01T10:00:08Z"
}
```

4. Domain Requirements

The implementation must use Domain-Driven Design principles.

The candidate must define and justify the domain model.

Expected concepts may include:

- Quote
- Conversion
- WalletAccount
- Money
- Asset
- ExchangeRate
- Reservation
- ConversionStatus

The candidate may choose a different model, provided the reasoning is clear.

4.1 Required Business Invariants

At minimum, the domain must enforce:

1. An expired quote cannot be accepted.
2. A quote can be accepted only once.

3. The same idempotency key cannot create multiple conversions.
4. Available balance cannot become negative.
5. Reserved balance cannot exceed total balance.
6. The same execution event cannot settle the wallet twice.
7. A completed conversion cannot return to a previous state.
8. A failed conversion must release its reservation.
9. Monetary values must not use JavaScript floating-point arithmetic.

Use one of the following approaches:

- Decimal library
 - Integer-based minor units
 - Another explicitly justified solution
-

5. Suggested Bounded Contexts

The candidate should keep the solution compact.

A reasonable design may contain:

Pricing Context

Responsibilities:

- Quote creation
- Exchange-rate calculation
- Quote expiration
- Quote acceptance rules

Wallet Context

Responsibilities:

- Balance
- Reservation
- Release
- Debit
- Credit
- Financial invariants

Conversion Context

Responsibilities:

- Conversion lifecycle
- Execution request
- Execution result
- State transitions

These contexts may exist inside one deployable modular monolith.

A microservice implementation is not required.

In fact, a modular monolith is preferred if it provides clearer transactional consistency and remains easier to review.

6. Recommended Architecture

A possible structure:

```
src/  
  modules/  
    pricing/  
      domain/  
      application/  
      infrastructure/  
      presentation/  
  
    wallet/  
      domain/  
      application/  
      infrastructure/  
  
    conversion/  
      domain/  
      application/  
      infrastructure/  
      presentation/  
  
    shared/  
      domain/  
      infrastructure/
```

The domain layer should not depend on:

- NestJS
- PostgreSQL
- RabbitMQ
- Redis

- HTTP
- ORM-specific entities

The candidate should avoid an anemic domain model in which all business rules are implemented inside services.

7. Technical Requirements

Required

- Node.js
- TypeScript
- NestJS
- PostgreSQL
- Docker Compose
- Unit tests
- Integration tests
- REST API
- Asynchronous event processing
- Outbox Pattern
- Idempotent consumer
- Structured logging

Messaging

The candidate may use:

- RabbitMQ, preferably
- An in-process message broker abstraction for the core implementation, provided a RabbitMQ adapter or design is included

Considering the candidate's claimed advanced RabbitMQ experience, practical RabbitMQ usage is preferred.

Optional

- Redis
- Prometheus metrics
- OpenTelemetry
- Swagger/OpenAPI
- Testcontainers
- Kubernetes manifests

Optional items should not replace correctness in transaction handling and domain design.

8. Concurrency Requirement

The candidate must demonstrate that concurrent quote acceptance cannot overspend the wallet.

Example initial state:

Wallet balance: 100 USDT

Two requests arrive concurrently:

Request A: reserve 80 USDT

Request B: reserve 80 USDT

Expected result:

One request succeeds.

One request fails.

The wallet balance never becomes negative.

The solution may use:

- Optimistic concurrency
- Conditional SQL update
- Row-level locking
- Serializable transaction
- Another justified database-level strategy

An in-memory mutex alone is not acceptable.

A Redis lock alone is not sufficient unless the database invariant is also protected.

9. Idempotency Requirement

The following repeated request:

POST /quotes/quote-001/accept

Idempotency-Key: key-123

must always return the same logical result.

It must not:

- Reserve funds twice
- Create two conversions
- Publish two logically distinct execution requests
- Settle the wallet twice

The candidate should explain:

- Where the idempotency record is stored
- What data is associated with the key
- Whether the request body is fingerprinted
- How concurrent duplicate requests are handled
- How long idempotency records are retained

10. Outbox Requirement

Accepting a quote must persist both:

- The business state change
- The integration event

inside one local database transaction.

Conceptually:

```
Accept Quote
|
+-- Reserve Wallet Balance
+-- Create Conversion
+-- Mark Quote Accepted
+-- Insert Outbox Event
|
+-- Commit
```

A background publisher must:

- Read unpublished outbox records
- Publish them
- Mark them as published
- Retry transient failures
- Avoid silently losing events

The candidate should explain the residual duplicate-delivery risk and how the consumer handles it.

11. Simulated External Exchange

Do not integrate with a real exchange.

Implement an adapter such as:

```
interface ExchangeExecutionPort {  
    execute(command: ExecuteConversionCommand): Promise<ExecutionResult>;  
}
```

The fake adapter should support:

- Successful execution
- Explicit failure
- Timeout or unknown result

A timeout must not automatically trigger a duplicate execution without checking idempotency or execution status.

The candidate should explain how this would be handled with a real provider using a unique client order ID.

12. Testing Requirements

At minimum, include tests for:

Domain Unit Tests

- Expired quote rejection
- Duplicate quote acceptance rejection
- Insufficient available balance
- Reservation success
- Reservation release
- Invalid conversion state transition
- Duplicate execution result handling

Integration Tests

- Atomic quote acceptance
- Outbox record creation

- Duplicate idempotency key
- Concurrent acceptance against the same wallet
- Consumer idempotency
- Failed execution releases funds

One Concurrency Test

The test must run two or more concurrent acceptance requests and prove that overspending does not occur.

13. Observability Requirements

Expose or document at least the following signals:

Logs

Structured logs should include:

- correlation ID
- quote ID
- conversion ID
- user ID
- event ID
- operation result
- error code

Do not log:

- access tokens
- secrets
- complete sensitive payloads

Metrics

At minimum, document or implement:

```
quote_created_total
quote_acceptance_total
quote_acceptance_failed_total
conversion_completed_total
conversion_failed_total
outbox_pending_count
outbox_publish_failure_total
```

```
execution_retry_total
```

Bonus metrics:

```
http_request_duration_seconds  
wallet_reservation_conflict_total  
event_processing_duration_seconds
```

Avoid high-cardinality labels such as user ID or conversion ID in Prometheus metrics.

14. AI-Assisted Development Requirement

The use of AI is expected and considered a positive part of the evaluation.

The candidate must not merely state that AI was used.

They must provide evidence of how AI was incorporated into the engineering process.

14.1 Required AI Deliverables

Create an `ai/` directory:

```
ai/  
  AI_USAGE.md  
  PROMPT_ARCHITECTURE.md  
  prompts/  
    01_domain_discovery.md  
    02_architecture_review.md  
    03_concurrency_review.md  
    04_test_generation.md  
    05_security_review.md  
  outputs/  
    selected-output-01.md  
    selected-output-02.md
```

14.2 AI_USAGE.md

This document must explain:

- Which AI model or tools were used
- Which development activities used AI
- Which generated suggestions were accepted

- Which suggestions were rejected
 - Which errors or hallucinations were detected
 - How generated code was verified
 - Where human engineering judgement overrode AI output
 - Whether AI improved speed, design quality, or test coverage
-

14.3 PROMPT_ARCHITECTURE.md

This is a major evaluation artefact.

The candidate must describe their prompt architecture.

It should include:

- Prompt objective
- System context
- Domain context
- Constraints
- Expected output structure
- Verification criteria
- Iteration strategy
- Context boundaries
- Use of examples
- Anti-hallucination controls
- Separation between exploration, implementation, review, and validation prompts

A strong submission should avoid one large generic prompt.

It should demonstrate a deliberate prompt chain.

Example:

```
Domain Discovery Prompt
    ↓
Domain Model Critique Prompt
    ↓
Architecture Decision Prompt
    ↓
Concurrency Risk Review Prompt
    ↓
Implementation Prompt
    ↓
Test Gap Analysis Prompt
    ↓
Security and Reliability Review Prompt
```

15. Required Prompt Engineering Evidence

At least five prompts must be submitted.

Prompt 1 — Domain Discovery

Purpose:

- Identify domain concepts
- Define ubiquitous language
- Propose aggregates
- Discover invariants
- Identify unclear business assumptions

The prompt should instruct the model not to generate code yet.

Prompt 2 — Architecture Review

Purpose:

- Critique module boundaries
 - Detect coupling
 - Detect infrastructure leakage into the domain
 - Evaluate aggregate size
 - Evaluate transactional boundaries
-

Prompt 3 — Concurrency and Failure Review

Purpose:

- Identify race conditions
- Analyse duplicate request handling
- Analyse message redelivery
- Analyse partial failure
- Review wallet safety

This is one of the most important prompts.

Prompt 4 — Test Design

Purpose:

- Generate a risk-based test matrix
 - Cover domain invariants
 - Cover transaction boundaries
 - Cover duplicate delivery
 - Cover concurrent requests
 - Avoid superficial controller-only tests
-

Prompt 5 — Security and Production Review

Purpose:

- Identify unsafe logging
 - Validate input boundaries
 - Review API abuse risks
 - Review retry behaviour
 - Review observability gaps
 - Review denial-of-service risks
-

16. Prompt Quality Expectations

The prompts will be evaluated based on:

- Clarity
- Context quality
- Explicit constraints
- Requested output format
- Defined reviewer role
- Verification steps
- Iterative refinement
- Appropriate scope
- Ability to reduce hallucination
- Ability to challenge the candidate's own design

Weak prompt:

Build a DDD Node.js application for a crypto exchange.

Stronger prompt:

Act as a principal backend architect reviewing a NestJS modular monolith.

Context:

The system accepts a short-lived conversion quote, reserves wallet funds, and publishes an execution request through an outbox.

Constraints:

- PostgreSQL is the source of truth.
- Wallet balance must never become negative.
- At-least-once message delivery is assumed.
- Domain code must not depend on NestJS or ORM types.
- The solution must remain small enough for a 15-hour coding challenge.

Tasks:

1. Identify the proposed aggregates and invariants.
 2. Detect incorrect transactional boundaries.
 3. Identify race conditions in quote acceptance.
 4. Challenge whether Wallet and Conversion belong in the same aggregate.
 5. Return findings grouped by Critical, Important, and Optional.
 6. Do not generate implementation code.
-

17. Architecture Documentation

Create:

```
docs/  
  ARCHITECTURE.md  
  DOMAIN_MODEL.md  
  DECISIONS.md  
  FAILURE_SCENARIOS.md
```

ARCHITECTURE.md

Include:

- System overview
- Module boundaries
- Dependency direction
- Transaction boundaries
- Event flow
- Outbox flow
- Deployment assumptions

DOMAIN_MODEL.md

Include:

- Ubiquitous language
- Aggregates
- Entities
- Value Objects
- Domain Events
- Invariants
- State transitions

DECISIONS.md

Record at least three Architecture Decision Records:

1. Wallet concurrency strategy
2. Outbox and consumer-idempotency strategy
3. Modular monolith versus microservices

FAILURE_SCENARIOS.md

Explain handling for:

- Quote expires during acceptance
- Duplicate HTTP request
- Concurrent wallet reservation
- Database commit failure
- Outbox publication failure
- Duplicate message delivery
- Exchange timeout
- Consumer crash after external execution
- Execution succeeds but acknowledgement fails
- Failed execution requires wallet release

18. Required Diagrams

Use Mermaid.

Context Diagram

```

flowchart LR
    Client --> API
    API --> Pricing
    API --> Conversion
    Conversion --> Wallet
    Conversion --> Outbox
  
```

```
Outbox --> Broker
Broker --> ExecutionWorker
ExecutionWorker --> FakeExchange
```

Sequence Diagram

Include the successful quote acceptance and execution flow.

State Diagram

Include the conversion lifecycle.

Suggested states:

```
PENDING
FUNDS_RESERVED
EXECUTION_REQUESTED
COMPLETED
FAILED
REQUIRES_RECONCILIATION
```

The candidate may propose a better state model.

19. Deliverables

The repository must include:

```
README.md
docker-compose.yml
.env.example
src/
test/
docs/
ai/
```

The README must contain:

- Setup instructions
- Run instructions
- Test instructions
- API examples
- Architecture summary
- Main trade-offs
- Known limitations
- Time spent

- AI tools used
 - Assumptions
-

20. Out of Scope

Do not implement:

- Real cryptocurrency exchange integration
- Blockchain deposits or withdrawals
- Full KYC
- Complete authentication server
- Frontend application
- Admin panel
- Order book
- Matching engine
- Market-data ingestion
- Multi-currency accounting system
- Kubernetes production deployment
- Full event-sourcing implementation
- Complex fee engine

The goal is depth, not breadth.

21. Evaluation Criteria

Area	Weight
Domain modelling and DDD	20%
Wallet correctness and concurrency	20%
Idempotency and failure handling	15%
Node.js and NestJS implementation quality	10%
PostgreSQL and transaction design	10%
Messaging and Outbox implementation	10%
Testing quality	5%
AI usage and prompt architecture	7%
Documentation and communication	3%

22. Evaluation Details

Excellent Submission

- Business rules live inside the domain model.
- Monetary values are modelled safely.
- Wallet invariants are protected at database level.
- Concurrent acceptance is tested.
- HTTP and message processing are idempotent.
- Outbox is implemented correctly.
- Domain is independent of NestJS and ORM.
- Failure scenarios are explicitly modelled.
- Prompts are structured, iterative, and critical.
- AI output is verified rather than copied.
- Trade-offs are documented clearly.

Acceptable Submission

- Core flow works.
- Basic DDD boundaries are present.
- Transactions and idempotency are mostly correct.
- Tests cover primary paths.
- AI usage is documented but not deeply structured.
- Some infrastructure leakage or design weakness exists.

Weak Submission

- CRUD-oriented service with DDD naming only.
- Business rules exist only in controllers or services.
- Wallet balance is handled by read-modify-write without protection.
- Floating-point values are used for money.
- Duplicate requests create duplicate conversions.
- Events may be lost due to direct dual write.
- Retry logic is unsafe.
- AI-generated code is included without validation.
- One generic prompt is presented as prompt engineering.

23. Interview Follow-Up Questions

After submission, ask the candidate:

1. Why did you choose these aggregate boundaries?
 2. Why is Wallet not part of the Conversion aggregate?
 3. Where exactly is the non-negative balance invariant enforced?
 4. What happens when two acceptance requests execute concurrently?
 5. What happens if the transaction commits but the message is not published?
 6. What happens if the consumer processes the event twice?
 7. What happens if the exchange succeeds but the worker crashes before saving the result?
 8. Why did you use optimistic or pessimistic concurrency?
 9. How would this design change under very high wallet contention?
 10. Which part of the AI-generated output was technically incorrect?
 11. Which prompt produced the most valuable result?
 12. How did you prevent the AI from overengineering the solution?
 13. Which design decision was made by you rather than the AI?
 14. What would you change before deploying this to Pooleno production?
 15. How would you integrate this module with the existing legacy NestJS application?
-

24. Submission Instructions

- Submit a public or private Git repository.
 - Include complete local execution instructions.
 - The project must run using Docker Compose.
 - The test suite must run with one documented command.
 - Include the AI and prompt-engineering artefacts.
 - Do not include secrets.
 - Mention the actual time spent.
 - Clearly identify incomplete areas.
-

Final challenge statement

Build a narrow but production-conscious conversion workflow in NestJS using Domain-Driven Design. Prioritise financial correctness, concurrency control, idempotency, reliable event delivery, and explicit failure handling. Use AI as an engineering assistant and provide a deliberate, reviewable prompt architecture that demonstrates how AI output was guided, challenged, validated, and refined.